

Practical 1 Analysis

Sorting Algorithm

1. Bubble Sort

Definition

Bubble Sort is a simple comparison-based sorting algorithm that repeatedly compares adjacent elements and swaps them if they are in the wrong order. After each pass, the largest unsorted element moves to its correct position.

Key Points

- Comparison-based sorting algorithm.
- Compares adjacent elements.
- Largest element is placed at the end after each pass.
- Stable sorting algorithm.
- In-place sorting algorithm.
- Easy to implement but inefficient for large datasets.

Complexity

Case	Time Complexity
Best	$O(n)$
Average	$O(n^2)$
Worst	$O(n^2)$
Space	$O(1)$

Output :-

```
><=====><
1.Selection Sort
2.Insertion Sort
3.Bubble Sort
4.Merge Sort
5.Quick Sort
6.Exit the Program..

Enter Your choice (1-6) :- 3

Printing the Array Elements Before Sorting: 1 , 9 , 33 , 53 , 63 ,

We are sorting the Array with the help of Bubble Sort.

Printing the Array Elements After Sorting: 1 , 9 , 33 , 53 , 63 ,
```

2. Selection Sort

Definition

Selection Sort repeatedly finds the smallest element from the unsorted portion of the array and places it at the beginning.

Key Points

- Comparison-based sorting algorithm.
- Selects the minimum element in each pass.
- Smallest element is placed at the beginning after each pass.
- Not a stable sorting algorithm.
- In-place sorting algorithm.
- Performs fewer swaps than Bubble Sort.

Complexity

Case	Time Complexity
Best	$O(n^2)$
Average	$O(n^2)$
Worst	$O(n^2)$
Space	$O(1)$

Output :-

```
Enter the size of array: 5
Enter the elements of array : 53
1
63
33
9

><=====><
1.Selection Sort
2.Insertion Sort
3.Bubble Sort
4.Merge Sort
5.Quick Sort
6.Exit the Program..

Enter Your choice (1-6) :- 1

Printing the Array Elements Before Sorting: 53 , 1 , 63 , 33 , 9 ,

We are sorting the Array with the help of Selection Sort.

Printing the Array Elements After Sorting: 1 , 9 , 33 , 53 , 63 ,
```

3. Insertion Sort

Definition

Insertion Sort builds the sorted array one element at a time by inserting each element into its correct position in the already sorted portion.

Key Points

- Comparison-based sorting algorithm.
- Builds a sorted portion from left to right.
- Efficient for nearly sorted arrays.
- Stable sorting algorithm.
- In-place sorting algorithm.
- Simple and efficient for small datasets.

Complexity

Case	Time Complexity
Best	$O(n)$
Average	$O(n^2)$
Worst	$O(n^2)$
Space	$O(1)$

Output :-

```
><=====><
1.Selection Sort
2.Insertion Sort
3.Bubble Sort
4.Merge Sort
5.Quick Sort
6.Exit the Program..

Enter Your choice (1-6) :- 2

Printing the Array Elements Before Sorting: 1 , 9 , 33 , 53 , 63 ,

We are sorting the Array with the help of Insertion Sort.

Printing the  Array Elements After Sorting: 1 , 9 , 33 , 53 , 63 ,
```

4. Merge Sort

Definition

Merge Sort is a Divide and Conquer algorithm that recursively divides the array into smaller parts, sorts them, and merges them to produce a sorted array.

Key Points

- Divide and Conquer algorithm.
- Divides the array recursively.
- Merges sorted subarrays.
- Stable sorting algorithm.
- Not an in-place sorting algorithm.
- Suitable for large datasets.

Complexity

Case	Time Complexity
Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n \log n)$
Space	$O(n)$

Output :-

```
><=====><
1.Selection Sort
2.Insertion Sort
3.Bubble Sort
4.Merge Sort
5.Quick Sort
6.Exit the Program..

Enter Your choice (1-6) :- 4

Printing the Array Elements Before Sorting: 1 , 9 , 33 , 53 , 63 ,

We are sorting the Array with the help of Merge Sort.

Printing the  Array Elements After Sorting: 1 , 9 , 33 , 53 , 63 ,
```

5. Quick Sort

Definition

Quick Sort is a Divide and Conquer algorithm that selects a pivot element and partitions the array into smaller and larger elements around the pivot.

Key Points

- Divide and Conquer algorithm.
- Uses a pivot element for partitioning.
- Very fast in practice.
- In-place sorting algorithm.
- Not a stable sorting algorithm.
- Performance depends on pivot selection.

Complexity

Case	Time Complexity
Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n^2)$
Space	$O(\log n)$

Output :-

```
><=====><
1.Selection Sort
2.Insertion Sort
3.Bubble Sort
4.Merge Sort
5.Quick Sort
6.Exit the Program..

Enter Your choice (1-6) :- 5

Printing the Array Elements Before Sorting: 1 , 9 , 33 , 53 , 63 ,

We are sorting the Array with the help of Quick Sort.

Printing the  Array Elements After Sorting: 1 , 9 , 33 , 53 , 63 ,
```

Comparison Table

Feature	Bubble Sort	Selection Sort	Insertion Sort	Merge Sort	Quick Sort
Technique	Comparison	Comparison	Comparison	Divide & Conquer	Divide & Conquer
Best Time	$O(n)$	$O(n^2)$	$O(n)$	$O(n \log n)$	$O(n \log n)$
Average Time	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n \log n)$	$O(n \log n)$
Worst Time	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n \log n)$	$O(n^2)$
Space Complexity	$O(1)$	$O(1)$	$O(1)$	$O(n)$	$O(\log n)$ (Average)
Stable	✔ Yes	✘ No	✔ Yes	✔ Yes	✘ No
In-place	✔ Yes	✔ Yes	✔ Yes	✘ No	✔ Yes
Adaptive	✔ Yes	✘ No	✔ Yes	✘ No	✘ No
Suitable For	Small datasets	Small datasets	Nearly sorted arrays	Large datasets	Large datasets

Algorithm	First Element Fixed (Ascending)	Stable	In-place
Bubble Sort	Largest element	✔	✔
Selection Sort	Smallest element	✘	✔
Insertion Sort	No fixed element (grows sorted prefix)	✔	✔
Merge Sort	No fixed element	✔	✘
Quick Sort	Depends on pivot	✘	✔